[1] In big-O analysis, aggregate running time refers to the expected performance over a big enough number of operations. For instance, if toss a (fair) coin once, we don't know if it's going to be head or tail; if we toss it twice, it's possible that we get once tail and once head, but it's far from being certain. If we repeat the experiment a million times, and the coin is fair and we don't do any tricks with it, then we will get head and tail (very) close to 50% of the times.

Likewise, for some algorithms, a single operation can take longer than expected, but if we run it on a high volume of data, the average running time will be predictable.

[2] At least in theory - low level implementation may vary between different programming languages.